1. Problem Title & Link

Title: LeetCode 443 : String Compression

Link: LeetCode 443 - String Compression

2. Problem Statement (Short Summary)

You are given an array of characters chars.

Compress the array using the following rules:

- Consecutive repeating characters are replaced by:
  - Character + frequency
- If frequency = 1, only store character
- Modify the array **in-place**
- Return the new length

In simple terms:

Convert:

["a","a","b","b","c","c","c"]

Into:

["a","2","b","2","c","3"]

Return:

6

3. Examples (Input → Output)

Example 1

Input:

chars=["a","a","b","b","c","c","c"]

Output:

6

Compressed array:

["a","2","b","2","c","3"]

Example 2

Input:

chars=["a"]

Output:

1

Compressed:

["a"]

Example 3

Input:

chars=["a","b","b","b","b","b","b","b","b","b","b","b","b"]

Output:

4

Compressed:

["a","b","1","2"]

Explanation:

Count:

b = 12

Store count digit-by-digit.

4. Constraints

1 <= chars.length <= 2000

chars[i] is English letter,

digit, or symbol

Special restrictions:

- Must modify array in-place
- Use O(1) extra space

5. Core Concept (Pattern / Topic)

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create another array:

1. Count repeating characters
2. Store compressed result separately

Time:

O(n)

Space:

O(n)

Not acceptable because extra space is used.

Optimized Thinking

Use two pointers:

- i → read pointer

- write → write pointer

Algorithm:

1. Start from current character
2. Count consecutive occurrences
3. Write character
4. If count >1:
    - Convert count to string
    - Write digits individually
5. Move to next group

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[a,a,b,b,c,c,c]

Processing:

read

a a b b c c c

↑

write

↓

Store:

a2

a2b2

a2b2c3

Final:

[a,2,b,2,c,3]

8. Pseudocode (Language Independent)

write=0

i=0

```
while i<n:

    current=chars[i]

    count=0

    while i<n and chars[i]==current:

        count++
        i++

    chars[write]=current
    write++

    if count>1:

        convert count to string

        for digit in count:

            chars[write]=digit
            write++

return write
```

9. Code Implementation

Python

```
class Solution:

    def compress(self, chars):

        write = 0
        i = 0

        while i < len(chars):

            current = chars[i]
```

```
        count = 0

        while i < len(chars) and chars[i] == current:
            count += 1
            i += 1

        chars[write] = current
        write += 1

        if count > 1:

            for digit in str(count):
                chars[write] = digit
                write += 1

    return write
```

Java

```
class Solution {

    public int compress(char[] chars) {

        int write=0;
        int i=0;

        while(i<chars.length){

            char current=chars[i];
            int count=0;

            while(i<chars.length &&
                  chars[i]==current){

                count++;
                i++;
            }
```

```
        chars[write]=current;
        write++;

        if(count>1){

          String frequency=
                String.valueOf(count);

          for(char c:
                frequency.toCharArray()){

            chars[write]=c;
            write++;
          }
        }
      }

      return write;
    }
}
```

10. Time & Space Complexity

Time Complexity:

O(n)

Reason:

Every character is visited once.

Space Complexity:

O(1)

Reason:

Only pointers are used.

11. Common Mistakes / Edge Cases

1. Forgetting multi-digit frequencies

Wrong:

12 → '12'

Stored as:

['1','2']

not:

['12']

2. Writing count when count=1

Wrong:

a1

Correct:

a

3. Forgetting write pointer increment

4. Missing single element case

["a"]

12. Detailed Dry Run (Step-by-Step Table)

Input:

[a,a,b,b,c,c,c]

Current	Count	Write Position	Array
a	2	0→2	[a,2]
b	2	2→4	[a,2,b,2]
c	3	4→6	[a,2,b,2,c,3]

Final:

Length=6

Compressed array:

[a,2,b,2,c,3]

13. Common Use Cases (Real-Life / Interview)

File compression

Data transmission optimization

Log storage reduction

Memory-efficient encoding

14. Common Traps (Important!)

1. Using extra array
2. Writing count for single occurrence
3. Missing multiple-digit count

4. Incorrect pointer movement

15. Builds To (Related LeetCode Problems)

Progression:

- LC 443 → String Compression
- LC 151 → Reverse Words in a String
- LC 6 → Zigzag Conversion
- LC 49 → Group Anagrams
- LC 316 → Remove Duplicate Letters

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Extra Array	O(n)	O(n)	Simple
String Builder	O(n)	O(n)	Extra memory
Two Pointers	O(n)	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

The read pointer scans groups of identical characters while the write pointer stores compressed output directly in the original array. Since each character group is processed once, the algorithm achieves in-place compression efficiently.

18. Variations / Follow-Up Questions

1. Compress full strings instead of character arrays
2. Decompress compressed strings
3. Apply compression to streaming data
4. Compress words instead of characters
5. Implement Run Length Encoding (RLE)
6.